Stochastic Scheduling in AI Coding Agents

A Paper-Style Framework for Theory, Control, and Evaluation

Author: Alfonso Cruz, AI assisted | **Date:** 2026-03-30

Abstract

AI coding agents are better modelled as stochastic, budget-constrained search systems than as deterministic programs. This document synthesizes a practical framework linking the behavioural intuition of variable-ratio reinforcement with an engineering model based on optimal stopping, context control, quota governance, and statistical evaluation. The probabilistic foundation is made explicit: pass@k is the CDF of a geometric process under independent trials, generalising to a non-homogeneous Bernoulli process when per-attempt success probability drifts due to context poisoning or verifier feedback. A dedicated section analyses the pass@k optimisation problem from three complementary perspectives: estimating and stabilising p; reasoning about the first-success distribution and expected cost; and choosing the attempt budget k under a token constraint. An operational protocol formalises the treatment of agents as bounded stochastic workers with scoped tasks, Plan Mode, checkpointed progress, and aggressive stopping. The document also includes an executable evaluation scaffold and illustrative pass@k graphs.

1. Theoretical Foundation

AI coding agents are not deterministic programs; they are **stochastic, budget-constrained search systems**. The engineering shift is therefore:

Deterministic software engineering $\longrightarrow$ **Probabilistic system management**

Traditional software follows explicit control flow with relatively stable cost. AI agents instead iterate through planning, action, verification, interpretation, and retry. The number of steps to success is not known in advance, and each step incurs cost without guaranteeing progress, in line with the general logic of sequential decision-making under uncertainty [1, 2].

1.1. The Skinner Box Analogy

The "gambling" sensation in agent usage can be explained by analogy with a **variable-ratio reinforcement schedule**. The analogy is behavioural, not literal: it captures uncertain reward timing rather than claiming that software agents are conditioned organisms [3].

- **Action:** an agent step, such as a prompt or tool call.
- **Reward:** task success or measurable progress.
- **Cost:** token consumption, latency, and compute.

Figure 1. Classical operant-conditioning apparatus used here as an analogy for variable-ratio reward timing. The figure is conceptual rather than evidential.

2. Probabilistic Foundations: Geometric Process and Non-Homogeneous Extension

Before engineering controls can be applied, the correct probabilistic model must be identified. The short answer is: **the geometric distribution and its non-homogeneous extension**.

2.1. Core Model: Independent Trials

Assume each run has an independent, constant success probability p. Let N denote the trial index of the first success. Then:

$$P(N = k) = (1 - p)^{k-1}\, p, \qquad k = 1, 2, \ldots$$

This is the **geometric distribution** with parameter p. The pass@k metric, as used throughout this document, is its cumulative distribution function:

$$\text{pass@k} = P(N \leq k) = 1 - (1 - p)^k$$

Key interpretive point. pass@k is not a distribution; it is the *cumulative success probability* of a geometric process. The two engineering questions it answers are structurally distinct:

Question	Model
When will the agent succeed?	Geometric distribution
Will it succeed within k attempts?	Geometric CDF (pass@k)

2.2. Non-Homogeneous Extension: Drifting p

In real agents, p is not constant across attempts. Context accumulates, verifier feedback may improve or degrade subsequent prompts, and the agent's effective capability shifts with each retry. The correct generalisation is a **non-homogeneous Bernoulli process**, where each attempt k carries its own success probability p_k:

$$P(\text{first success at attempt } k) = p_k \prod_{i=1}^{k-1}(1 - p_i)$$

The pass@k generalisation follows directly:

$$P(\text{success within } k \text{ attempts}) = 1 - \prod_{i=1}^{k}(1 - p_i)$$

This model makes the connection to the rest of the framework precise:

- **Thrashing** inflates $(1 - p)$, flattening the pass@k curve.
- **Context poisoning** (see §3.2) causes $p_k \downarrow$ as k increases, making later attempts actively cheaper to abort.
- **Good planning and verifier feedback** cause $p_k \uparrow$, turning the retry sequence into an improving process rather than an i.i.d. gamble.

> **One-liner for the paper.** The pass@k metric corresponds to the CDF of a geometric process under independent trials, and generalises to a non-homogeneous Bernoulli process when agent performance evolves across iterations due to context drift, verifier feedback, or prompt adaptation.

2.3. Practical Diagnostic: Is Your Agent Homogeneous?

To determine which model applies, compute the per-position empirical success rates $\hat{p}_1, \hat{p}_2, \ldots, \hat{p}_k$ from repeated runs and inspect the sequence:

- **Stationary $\hat{p}_k$:** the i.i.d. geometric model holds; the simple formula $1 - (1 - p)^k$ is valid.
- **Declining $\hat{p}_k$:** context poisoning is active; abort early and restart from a clean context.
- **Rising $\hat{p}_k$:** verifier feedback is working; allow more attempts and invest in the feedback loop.

This diagnostic is the empirical bridge between the probabilistic model and the engineering controls in §7.

3. Formal Operational Model

Let T denote tokens spent and let $P(\text{success} \mid T)$ denote the probability of success given that expenditure. Execution should continue only while marginal expected utility remains acceptable.

3.1. Stopping Rule (Abstinence Logic)

A minimal stopping rule is

$$\frac{dP}{dT} > \tau$$

where τ is a policy threshold.

A more general utility-based form is

$$\frac{dU}{dT} > 0$$

where U can incorporate success probability, latency penalties, and quota pressure. This is an optimal-stopping style control rule rather than a mere heuristic retry loop. In the non-homogeneous setting of §2.2, P at attempt k is a function of the sequence $p_1, \dots, p_k$, making the marginal gain of one more attempt estimable from the empirical trend in $\hat{p}_k$.

3.2. Context Poisoning

As a dialogue grows, token cost often scales superlinearly. For full-attention transformer models operating over a monolithic context, a useful approximation is

$$T(n) = O(n^2)$$

for iterative depth n. Note that this bound is model-dependent: architectures employing sliding-window attention, sparse attention, or KV-cache eviction may exhibit different scaling behaviour. In practice, context accumulation can reduce signal-to-noise ratio and eventually degrade performance instead of improving it — manifesting as a declining p_k sequence in the non-homogeneous model. This is the operational reason for summarization, pruning, checkpointing, and bounded retries.

4. The Reliability Surface

Evaluation must move beyond single-run pass/fail judgment toward a multi-dimensional **Reliability Surface**.

Metric	Definition	Purpose
Pass@k	$1-(1-p)^k$ (i.i.d. runs; see §9.3)	CDF of geometric process; probability of at least one success in k attempts.
Cost Stability	σ/μ	Predictability of token cost (std. dev. over mean).
Thrash Score	repeated_steps/total_steps	Detects looping; inflates effective $(1-p)$.
Fragility (ΔS)	Var(Output)/Var(Input)	Sensitivity to small input perturbations.

These metrics should be interpreted jointly rather than in isolation, consistent with broader systems thinking about performance distributions and tail behaviour [4, 5].

5. Illustrative pass@k Graphs

The following figures are illustrative. They are not empirical measurements from a specific benchmark unless replaced with your own generated plots.

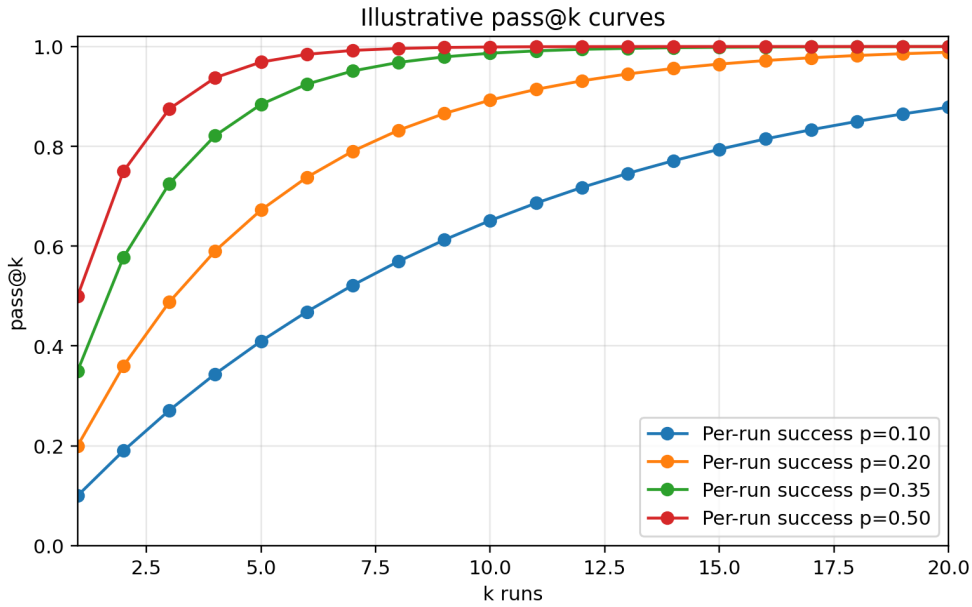

Figure 2. Illustrative pass@k curves (geometric CDF) for several hypothetical per-run success probabilities p.

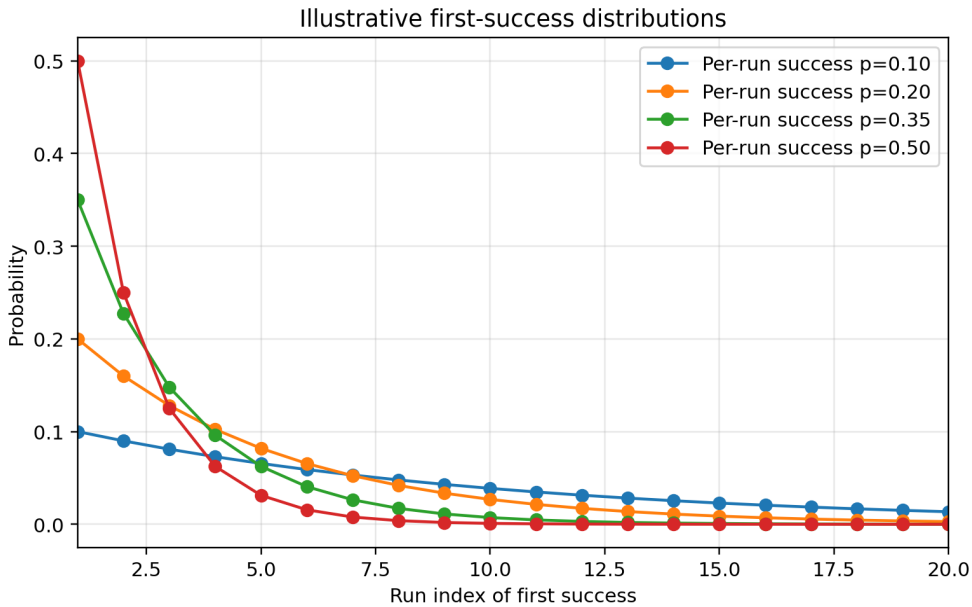

Figure 3. Illustrative distributions of the run index N at which the first success occurs, i.e. the geometric PMF with parameter p.

6. pass@k Optimisation: Three Perspectives

The pass@k formula $1-(1-p)^k$ depends on exactly two quantities: the per-run success probability p and the attempt budget k. Optimising pass@k therefore reduces to three distinct, complementary problems that should be addressed in order: *stabilise* p first, then *reason about cost to first success*, then *choose* k *under a budget constraint*.

6.1. Perspective 1: Estimating and Stabilising p

The formula $1-(1-p)^k$ is only as meaningful as the estimate of p feeding into it. A single observed success or failure is a poor estimator; p must be treated as a latent variable to be inferred from repeated runs.

Point estimation. The maximum-likelihood estimate is simply the empirical success rate:

$$\hat{p} = \frac{c}{n}$$

where n is the number of runs and c the number of successes. This is unbiased but high-variance for small n.

Interval estimation. For operational decisions, a conservative lower confidence bound is more useful than a point estimate. Using a Wilson score interval at confidence level $1-\alpha$:

$$p_{\text{low}} = \frac{\hat{p} + \frac{z^2}{2n} - z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}$$

where $z = z_{1-\alpha/2}$ is the standard normal quantile. Planning with p_{low} instead of $\hat{p}$ builds in a safety margin against optimistic over-counting.

Stabilising p in practice. A high pass@k achieved by increasing k masks a low p. The more cost-effective lever is to raise p itself. Techniques that reliably increase p include:

- **Prompt hardening** — structured system prompts, explicit output contracts, and few-shot exemplars reduce the variance of the output distribution.
- **Temperature scheduling** — a lower temperature on the first attempt and a higher temperature on retries widens coverage without sacrificing first-attempt precision.
- **Context pruning before retry** — removing accumulated noise from the context window before each attempt prevents context poisoning (§3.2) from compounding across runs, and arrests a declining p_k sequence.
- **Verifier feedback** — passing the failure mode from an automated test or linter back into the prompt as a structured hint raises the conditional success probability p_k on subsequent attempts, producing a rising sequence and warranting additional attempts.

Checking the i.i.d. assumption. The formula $1-(1-p)^k$ assumes that runs are independent and identically distributed. As established in §2.2, both assumptions are routinely violated. A practical check is to compute the per-position empirical success rates $\hat{p}_1, \hat{p}_2, \ldots, \hat{p}_k$ and test whether

they are stationary. A declining sequence signals context poisoning or prompt drift; a rising sequence signals that verifier feedback is working. In neither case is the homogeneous geometric model appropriate, and the non-homogeneous formula should be used for budget projections.

6.2. Perspective 2: First-Success Distribution and Expected Cost

The number of attempts required to obtain the first success follows the geometric distribution established in §2:

$$P(N = k) = (1 - p)^{k-1}\, p$$

Expected attempts.

$$\mathbb{E}[N] = \frac{1}{p}$$

This is the most direct cost-facing quantity: the average number of agent runs — and therefore the average token spend — required to obtain one successful result. Doubling p halves expected cost; this is the return on any investment in prompt hardening.

Expected token cost. Let $\bar{T}$ be the mean tokens consumed per run. The expected total token cost to first success is:

$$\mathbb{E}[\text{cost to success}] = \frac{\bar{T}}{p}$$

This gives a single actionable number for quota planning. Setting a token budget B and solving for the required p:

$$p^* \geq \frac{\bar{T}}{B}$$

If $\hat{p} < p^*$, the prompt must be hardened before scaling k.

Tail behaviour. The probability of requiring more than m attempts is:

$$P(N > m) = (1 - p)^m$$

For operational SLAs, the attempt ceiling k_{max} such that $P(N > k_{\text{max}}) \leq \epsilon$ is:

$$k_{\text{max}} \geq \frac{\ln \epsilon}{\ln(1 - p)}$$

This translates a reliability target directly into an attempt budget.

6.3. Perspective 3: Choosing k Under a Token Budget

Given an estimated p, a mean per-run token cost $\bar{T}$, and a total token budget B, the maximum feasible attempt count is:

$$k_{\text{max}} = \left\lfloor \frac{B}{\bar{T}} \right\rfloor$$

The resulting pass@k under this budget is:

$$\text{pass@}k_{\text{max}} = 1 - (1 - p)^{\lfloor B/\bar{T} \rfloor}$$

Optimisation under a reliability target. If the goal is to hit a target pass@k of R with minimum token spend, the required k is:

$$k^* = \left\lceil \frac{\ln(1-R)}{\ln(1-p)} \right\rceil$$

and the corresponding minimum budget is $k^* \cdot \bar{T}$.

Jointly optimising p and k. Spending tokens on prompt engineering (raising p) competes with spending tokens on additional attempts (raising k). Given a fixed total budget B split as $B = B_p + k\bar{T}$, the joint optimisation problem is:

$$\max_{B_p,\, k \geq 1} \left[1 - (1 - p(B_p))^k\right] \quad \text{s.t.} \quad B_p + k\bar{T} \leq B$$

where $p(B_p)$ is a monotone non-decreasing function mapping engineering investment to per-run success rate. If p is already high, additional prompt investment yields diminishing returns and increasing k is more efficient; if p is low, raising k is a costly substitute for a fundamentally broken prompt.

Adaptive stopping. Rather than committing to a fixed k upfront, an adaptive policy updates the estimate of p after each run and stops when either (a) a success is observed, or (b) the posterior expected gain of one more attempt falls below the marginal token cost. This connects to the stopping rule in §3.1 and transforms the static pass@k formula into a sequential decision process over the geometric distribution.

> **Policy summary.** In order of priority: (1) measure and stabilise p; (2) use $\mathbb{E}[\text{cost}] = \bar{T}/p$ for budget planning; (3) choose $k^* = \lceil \ln(1-R)/\ln(1-p) \rceil$ to hit a reliability target at minimum spend; (4) apply adaptive stopping to avoid wasting budget on runs whose expected marginal gain is negative; (5) switch to the non-homogeneous model if $\hat{p}_k$ is not stationary.

7. Operational Protocol for Bounded Agentic Workers

The most intelligent way to operate an AI coding agent is to treat it as a **bounded stochastic worker**, not as a brilliant intern permitted to roam freely. The winning pattern is tight task scope, plan before execution, small context, checkpointed progress, and aggressive stopping when marginal gains fall. This section formalises that pattern as an operational protocol compatible with Anthropic's own guidance on Plan Mode, context management, and long-running agent harnesses.

7.1. Task Scoping: Atomic Episodes

Split work into short, atomic episodes. For each episode, specify:

1. One concrete objective.
2. The acceptance test (definition of done).
3. The files or directories in scope.
4. The exact output format required.

Asking for "solve this whole project" generates vague plans, wide diffs, and expensive repair turns. A precise task contract often saves more tokens than any model or parameter tweak [9].

7.2. Plan Mode First

For each episode, run in Plan Mode before any implementation. Review the proposed plan and approve only the first bounded implementation step. Anthropic explicitly recommends Plan Mode for exploring codebases and planning complex changes safely; it also reduces the probability of a bad first attempt, directly raising p_1 in the non-homogeneous model [7].

7.3. Context Discipline and Checkpointing

Long conversations become more expensive and less reliable as they approach the context window. Active context management is required for long-running sessions [8]. The recommended practice is:

- After each meaningful milestone, collapse state into a short written checkpoint in the repository or task file.
- Begin the next episode from that artifact, not from a bloated chat history.
- Never let one thread become the system of record. Put state in files, not in the conversation.

This directly arrests the $p_k \downarrow$ trend caused by context poisoning (§3.2) by resetting the context window at each checkpoint.

7.4. The Spec–Plan–Patch–Verify Workflow

The highest-efficiency pattern is:

1. **Spec:** give a scoped brief — goal, constraints, files, tests, definition of done.
2. **Plan:** run Plan Mode; review the plan before any changes.
3. **Patch:** implement one bounded step only.
4. **Verify:** run the specified tests; write a checkpoint note summarising what changed.
5. **Stop or continue:** start the next step from the checkpoint when the context begins to swell. Stop when the next step is no longer clearly higher-value than its token cost (i.e. when the stopping rule in §3.1 fires).

This workflow reduces thrashing, keeps diffs reviewable, and makes failures cheap. The goal is fewer bad turns, not maximum autonomy: a single vague prompt can generate ten expensive repair turns [9].

7.5. Thinking Budget: Deliberate, Not Default

Extended or adaptive thinking improves instruction-following, reasoning, and efficiency in agent systems [10]. Use it deliberately:

- **Heavy thinking:** hard planning, ambiguous debugging, architectural decomposition — tasks where the question is "what should we do?"

- **Light execution:** trivial edits, formatting, straightforward refactors — tasks where the answer is "do this exact thing."

Paying the thinking cost for routine execution is a direct waste of the token budget B without a commensurate increase in p.

7.6. Protocol Summary

Default workflow.

1. Give a scoped brief: goal, constraints, files, tests, definition of done.
2. Run **Plan Mode** first.
3. Approve only the first bounded implementation step.
4. Make the agent write a short checkpoint note after each step.
5. Start the next step fresh from that checkpoint when context swells.
6. Stop when the next step is no longer clearly higher-value than its token cost.

The most important rule: **optimise for fewer bad turns, not for maximum autonomy.**

8. Engineering Implementation Best Practices

To transform stochastic behaviour into controlled execution, engineering teams should combine the operational protocol of §7 with the following implementation-level practices. Trajectory review in particular has been shown to be effective when agents interleave reasoning and acting steps [6].

- **Review trajectories, not just code.**
- **Enforce idempotency** for tool calls and external side effects.
- **Checkpoint state** every n steps or major milestone.
- **Apply token rationing** through rolling-window or bucket-based limits.

9. Minimal Executable Scaffold

The framework becomes more useful when paired with a minimal, reproducible evaluation pipeline.

9.1. Logging Schema

Listing 1. Minimal run-level logging schema.

```
{
  "task_id": "task_001",
  "run_id": "run_01",
  "steps": 12,
  "tokens": 18432,
  "success": true,
  "repeated_steps": 3,
```

```
  "unique_states": 9,
  "max_context_tokens": 22000,
  "latency_sec": 48.2
}
```

9.2. Core Metric Code

Listing 2. Metric functions and pass@k optimisation helpers.

```python
import numpy as np
from scipy import stats

#   Reliability surface metrics

def cost_stability(tokens):
    return np.std(tokens) / np.mean(tokens)

def tail_risk(tokens):
    return np.percentile(tokens, 95) / np.median(tokens)

def thrash_score(repeated_steps, total_steps):
    return repeated_steps / total_steps

def progress_score(unique_states, total_steps):
    return unique_states / total_steps

#   Geometric model (homogeneous)

def pass_at_k_from_p(p, k):
    """Geometric CDF: P(first success within k attempts)."""
    return 1.0 - (1.0 - p) ** k

def geometric_pmf(p, k):
    """P(first success on exactly attempt k)."""
    return (1.0 - p) ** (k - 1) * p

#   Non-homogeneous Bernoulli process

def nonhomogeneous_pass_at_k(p_seq):
    """P(success within len(p_seq) attempts) for drifting p_k.

    p_seq: list of per-attempt success probabilities [p_1, p_2, ..., p_k]
    """
    return 1.0 - np.prod([1.0 - p for p in p_seq])

def nonhomogeneous_pmf(p_seq, k):
    """P(first success on attempt k) under non-homogeneous model."""
    if k > len(p_seq):
        raise ValueError("k exceeds length of p_seq")
    survival = np.prod([1.0 - p_seq[i] for i in range(k - 1)])
    return survival * p_seq[k - 1]

#   pass@k optimisation helpers

def p_hat(successes, runs):
    """MLE estimate of per-run success probability."""
```

```python
    return successes / runs

def p_wilson_lower(successes, runs, confidence=0.95):
    """Conservative lower Wilson score bound for p."""
    alpha = 1 - confidence
    z = stats.norm.ppf(1 - alpha / 2)
    ph = successes / runs
    denom = 1 + z**2 / runs
    centre = ph + z**2 / (2 * runs)
    margin = z * np.sqrt(ph * (1 - ph) / runs + z**2 / (4 * runs**2))
    return (centre - margin) / denom

def expected_cost_to_success(p, mean_tokens_per_run):
    """E[total tokens to first success] = mean_T / p."""
    return mean_tokens_per_run / p

def min_p_for_budget(budget, mean_tokens_per_run):
    """Minimum p required so that E[cost] <= budget."""
    return mean_tokens_per_run / budget

def k_for_target_reliability(p, target_pass_at_k):
    """Minimum k to achieve target pass@k given p."""
    return int(np.ceil(np.log(1 - target_pass_at_k) / np.log(1 - p)))

def k_max_from_budget(budget, mean_tokens_per_run):
    """Maximum attempt count affordable within budget."""
    return int(np.floor(budget / mean_tokens_per_run))

def attempt_ceiling_for_sla(p, epsilon):
    """k_max s.t. P(need > k_max attempts) <= epsilon."""
    return int(np.ceil(np.log(epsilon) / np.log(1 - p)))

#   Stationarity diagnostic

def per_position_success_rate(results_matrix):
    """Compute empirical p_k for each attempt position.

    results_matrix: 2-D boolean array, shape (n_tasks, k_max).
    Only the first True in each row counts as a success.
    Returns array of shape (k_max,).
    """
    n_tasks, k_max = results_matrix.shape
    rates = []
    for k in range(k_max):
        # eligible: tasks that have not yet succeeded before attempt k
        eligible = ~results_matrix[:, :k].any(axis=1)
        if eligible.sum() == 0:
            rates.append(float('nan'))
        else:
            rates.append(results_matrix[eligible, k].mean())
    return np.array(rates)
```

9.3. Notes on Estimation

For repeated-run operational measurements, a simple empirical model is often sufficient:

$$\text{pass@k} = 1 - (1 - p)^k$$

where p is the observed per-run success rate. This form assumes i.i.d. runs; verify stationarity with `per_position_success_rate` before applying it.

For code-generation benchmark settings with finite samples, an unbiased estimator is often preferred:

$$\text{pass@k} = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}$$

where n is the number of samples and c the number of correct samples.

9.4. Python Utility

The bundle includes a runnable script named `plot_pass_distributions.py`. It can generate:

- illustrative pass@k figures,
- empirical pass@k curves from a CSV of repeated runs,
- token distribution histograms,
- a summary CSV per agent.

10. Industry Reality Check

Serious teams already use fragments of this framework in practice: multi-run evaluation, timeouts, retry limits, and manual inspection of failures. What remains uncommon is a unified treatment that: (a) grounds pass@k in its correct probabilistic model — the geometric distribution and its non-homogeneous generalisation; (b) connects the metric to its engineering levers through the three-perspective optimisation framework of §6; and (c) pairs it with an explicit operational protocol that treats the agent as a bounded stochastic worker rather than an autonomous solver.

11. Conclusion

AI agents are not functions to evaluate once, but **stochastic systems to evaluate statistically and control rigorously**. The pass@k metric is the CDF of a geometric process; when agent performance drifts across attempts it generalises to a non-homogeneous Bernoulli process. The most intelligent policy is: stabilise p first, plan cost using $\bar{T}/p$, choose k analytically to meet a reliability target at minimum spend, and operate the agent as a bounded stochastic worker with tight task scope, Plan Mode, and aggressive context checkpointing.

References

[1] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, second edition, 2018.

[2] Leslie Pack Kaelbling, Michael L. Littman, and Andrew W. Moore. Reinforcement learning: A survey. *Journal of Artificial Intelligence Research*, 4:237–285, 1996.

[3] Charles B. Ferster and B. F. Skinner. *Schedules of Reinforcement*. Appleton-Century-Crofts, 1957.

[4] Jeffrey Dean and Luiz André Barroso. The tail at scale. *Communications of the ACM*, 56(2):74–80, 2013.

[5] Joseph L. Hellerstein, Yixin Diao, Sujay Parekh, and Dawn M. Tilbury. *Feedback Control of Computing Systems*. Wiley-IEEE Press, 2004.

[6] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. arXiv preprint arXiv:2210.03629, 2022.

[7] Anthropic. Common workflows — Claude Code documentation. https://docs.anthropic.com/en/docs/claude-code/common-workflows, 2025.

[8] Anthropic. Context windows — Claude API documentation. https://docs.anthropic.com/en/docs/build-with-claude/context-windows, 2025.

[9] Anthropic. Prompt engineering overview — Claude API documentation. https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/overview, 2025.

[10] Anthropic. How we built our multi-agent research system. https://www.anthropic.com/engineering/multi-agent-research-system, 2025.